

Laser: Unlocking Layer-Level Scheduling for Efficient Multi-SLO LLM Serving

Jianxiong Liao
Sun Yat-sen University
Guangzhou, China
liaojx9@mail2.sysu.edu.cn

Quanxing Dong
Sun Yat-sen University
Guangzhou, China
dongqx3@mail2.sysu.edu.cn

Yunkai Liang
Sun Yat-sen University
Guangzhou, China
liangyk7@mail2.sysu.edu.cn

Zhi Zhou*
Sun Yat-sen University
Guangzhou, China
zhouzhi9@mail.sysu.edu.cn

Xu Chen*
Sun Yat-sen University
Guangzhou, China
chenxu35@mail.sysu.edu.cn

Abstract

Engaging applications with diverse SLO requirements has become indispensable for production-scale LLM serving systems. However, existing systems rely on iteration-level scheduling, which enforces inflexible, unified execution across multi-SLO workloads, significantly constraining the serving efficiency.

In this paper, we introduce layer-level scheduling, a novel mechanism that advances beyond conventional iteration-level granularity. This mechanism decomposes per-iteration computation into fine-grained layer operations, enabling the tailored execution of requests with differing requirements. However, this increased granularity introduces new challenges in both intra-instance request execution and cross-instance coordination, posing significant barriers to practical deployment. To address these challenges, we introduce Laser, a system designed for efficient multi-SLO LLM serving. The key aspect lies in the seamless integration of inter-instance request dispatching with layer-level scheduling within instances, delivering high serving throughput with SLO guarantees. Evaluations with real-world applications reveal that Laser effectively improves throughput by over $1.67\times$ while maintaining the same SLO attainment rate compared to state-of-the-art systems.

CCS Concepts: • Computer systems organization → Real-time systems; • Computing methodologies → Machine learning approaches.

Keywords: Layer-level Scheduling, Multi-SLO Serving, Large Language Model

*Corresponding authors: Zhi Zhou, Xu Chen.



This work is licensed under a Creative Commons Attribution 4.0 International License.

PPoPP '26, Sydney, NSW, Australia

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2310-0/2026/01

<https://doi.org/10.1145/3774934.3786413>

ACM Reference Format:

Jianxiong Liao, Quanxing Dong, Yunkai Liang, Zhi Zhou, and Xu Chen. 2026. Laser: Unlocking Layer-Level Scheduling for Efficient Multi-SLO LLM Serving. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '26)*, January 31 – February 4, 2026, Sydney, NSW, Australia. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3774934.3786413>

1 Introduction

Large Language Models (LLMs) [7, 17, 31, 36, 38, 42] have achieved state-of-the-art performance across a wide spectrum of domains, ranging from natural language processing, code generation, and multimodal reasoning. Built upon transformer architectures [37], these models exhibit remarkable scalability — their capabilities consistently improve as the number and width of transformer layers increase. These advancements have sparked the integration of LLMs into production systems, where they enhance functionality and deliver improved user experiences.

The core computational process of LLMs lies in autoregressive generation, which produces output tokens sequentially. This process naturally splits LLM serving into two distinct phases: (1) **Prefill**: The initial phase that processes the entire input prompt in parallel to generate the first token. (2) **Decode**: The subsequent, sequential phase that generates tokens one-by-one. To accommodate the iterative process, continuous batching [41] is proposed to flexibly batch requests at the granularity of individual iterations¹, greatly improving GPU utilization and serving throughput. Built on this iteration-level mechanism, recent advances [20, 28, 30, 43] further develop a disaggregated architecture that isolates prefill and decode phases onto dedicated hardware, allowing phase-specific optimization to meet the distinct resource demands and performance metrics. These advances have become mainstream enablers for efficient LLM serving at production scale [3, 23, 30].

¹In this work, we define an iteration as one full forward pass through all model layers.

Recently, the remarkable generalizability of LLMs has driven a shift toward hosting diverse applications on shared foundation models in the cloud. For instance, Alibaba deploys a centrally maintained Qwen 3 model [4, 22, 40] to support a unified suite of agents, including a Reading Assistant for comprehension, a Code Completion tool for developers, and an Image Generator for creators, offering a cohesive user experience. To maintain responsiveness for online users, these applications typically operate under strict service-level objectives (SLOs). However, the SLO requirements vary widely across scenarios and user expectations. Conversational chatbots [17, 38], for example, demand fast prefill to preserve interactivity but favor steadier, human-like decoding speeds for natural dialogue. In contrast, productivity tools [10] emphasize fast generation in both prefill and decode phases to maximize efficiency.

Existing systems predominantly [6, 19, 23] incorporate the iteration-level scheduling mechanism, executing requests at the granularity of entire iterations. However, this limited granularity presents fundamental limitations in sustaining high throughput while maintaining SLO compliance under multi-SLO workloads. In the prefill phase, it lacks the agility to dynamically chunk and switch request execution, causing head-of-line blocking that violates SLOs of latency-critical requests and limited chunk-size execution that extends prefill latency of relaxed requests. In the decode phase, it batches requests with heterogeneous SLOs uniformly, leading to a mismatch between the identical decode time and the multi-SLO requirements. This constrains the capacity of instances to accommodate requests with relaxed requirements, ultimately leading to more SLO violations.

These limitations motivate the exploration of **layer-level scheduling**, a novel mechanism built on two key innovations: (1) **Layer-level chunked prefill**, which promptly prioritizes latency-critical requests with efficient request switching while dynamically consolidating relaxed requests into larger chunks within iterations. (2) **Layer-level decode batching**, which precisely controls the number of layers executed per iteration for each request, tailoring execution to its specific SLO target. Together, these techniques enable the system to independently satisfy the heterogeneous demands of multi-SLO workloads, simultaneously guaranteeing strict SLO attainment and high overall throughput.

However, the practical implementation of layer-level scheduling introduces substantial design challenges. First, the variability in per-layer execution complicates latency evaluation of individual iterations and significantly expands the search space, hindering efficient request scheduling within instances. Second, dispatching requests to instances necessitates careful consideration of layer-level runtime states to accurately assess the impact of request assignment and prevent SLO violations. In response to these challenges, we present Laser, an efficient serving system for multi-SLO workloads

on top of layer-level scheduling. Laser incorporates a dual-level design with the prefill-decode disaggregation architecture. At the intra-instance level, Laser adopts efficient, phase-specific scheduling algorithms built on precise layer-level latency modeling to optimize serving latency with SLO guarantees. At the inter-instance level, Laser seamlessly coordinates request dispatching with intra-instance scheduling by incorporating real-time scheduling outcomes from each instance, enabling holistic optimization across multi-SLO workloads.

We have implemented a prototype of Laser atop vLLM [23] and evaluated its design through extensive experiments across different models and workloads. Experimental results demonstrate that Laser improves goodput by over 1.67× compared to state-of-the-art systems. In summary, we make the following contributions:

- We comprehensively analyze the limitations of iteration-level scheduling and propose layer-level scheduling that enables granular control of request execution within iterations to handle the heterogeneous requirements of multi-SLO workloads.
- We design and implement Laser, a serving system that realizes layer-level scheduling through the integration of intra- and inter-instance optimizations.
- We conduct evaluations across a wide range of scenarios, demonstrating the substantial improvement of Laser compared to state-of-the-art systems.

2 Background

2.1 LLM Architecture

Large language models (LLMs) are predominantly built on the Transformer architecture [37], which consists of a stack of transformer layers. Each layer processes the intermediate states from its predecessor and forwards the result, while the output of the final layer is projected into a probability distribution over the vocabulary to generate output tokens. To optimize computational efficiency, LLM serving systems cache intermediate token states, known as the KV cache [23], in GPU High Bandwidth Memory (HBM). This mechanism avoids redundant computations by reusing cached states during subsequent token generation.

2.2 Iteration-level Request Scheduling

Unlike traditional DNN inference that completes computation in a single forward pass [18, 35], modern LLMs [36, 40, 42] operate autoregressively, generating one output token for each iteration until reaching the maximum sequence length or emitting an end-of-sequence (EOS) token. This iterative process significantly extends the end-to-end latency of individual requests. Consequently, traditional request-level scheduling, where new requests wait until prior ones finish, cannot expand batch sizes during long-running executions,

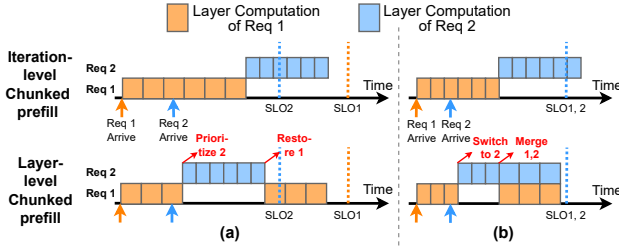


Figure 1. Comparison between different scheduling approaches in the prefill phase.

resulting in inefficient parallelism and poor resource utilization. Orca [41] mitigates this inefficiency through *continuous batching*, which shifts the batching granularity from requests to iterations. This approach allows new requests to be flexibly integrated at each iteration, irrespective of the variability in request lengths, yielding significant improvements in throughput and resource utilization. Sarathi-Serve [6] introduces *chunked prefill* on top of this technique, partitioning long prefill computations into smaller chunks executed across iterations to prevent prefill from blocking decode execution. These iteration-level scheduling mechanisms have become the de facto standard for efficient LLM serving.

2.3 Prefill-Decode Disaggregation

The autoregressive generation process creates two distinct computational phases in LLM serving. The prefill phase processes the entire input sequence to produce the first output token, with the latency captured by Time-to-First-Token (TTFT). This phase is compute-bound, leveraging the computational power of accelerators (e.g., GPUs) to parallelize batched matrix operations. In contrast, the decode phase generates subsequent tokens iteratively and processes only the most recent token for each iteration, with the latency measured by Time-Between-Tokens (TBT). This phase is memory-bound, constrained by frequent KV cache accesses and sequential execution.

To meet the distinct resource and latency requirements of prefill and decode phases, recent LLM serving systems [20, 28, 30, 43] introduce the innovative prefill–decode disaggregation architecture, deploying specialized serving instances for each phase. Incoming requests are initially processed by prefill instances, which generate the first token while building the complete KV cache. The populated KV cache is then migrated to decode instances, which take over the iterative generation of subsequent tokens. By physically separating these phases, this architecture eliminates inter-phase contention and enables targeted optimizations that precisely satisfy the specific resource and SLO requirements of each stage.

2.4 Multi-SLO LLM Serving

The rapid growth in both the scale and architectural sophistication of LLMs has enabled the emergence of diverse,

specialized applications built atop shared foundation models. Each application typically exhibits distinct SLO requirements, shaped by its unique interaction patterns and user expectations. For example, conversational agents like ChatBots [38] emphasize low TTFT for immediate perceived responsiveness while tolerating more relaxed TBT requirements aligned with human reading speeds (e.g., 10 tokens per second [9]). Conversely, productivity tools such as coding assistants require stringent TBT thresholds below 50 ms to maintain seamless developer workflows [12]. The landscape grows increasingly complex with advanced techniques [17, 39], which can introduce varied SLO requirements even within a single application. This heterogeneity in performance requirements highlights the critical need for efficient, SLO-aware LLM serving systems to deliver optimal performance across diverse application workloads.

3 Limitations and Proposed Solutions

In this section, we examine the limitations of iteration-level scheduling in handling multi-SLO workloads and introduce two phase-specific techniques: layer-level chunked prefill and layer-level decode batching.

L1: Inflexible prefill chunking. In the prefill phase, the chunk size used in chunked prefill critically determines both iteration latency and average latency per token. As Fig. 2a shows, increasing the chunk size from 100 to 3200 raises per-iteration latency by 16× but reduces per-token latency by 45.4%, indicating that larger chunk sizes can significantly improve serving throughput at the expense of higher prefill iteration overhead. Existing systems that incorporate iteration-level chunked prefill are hampered by their inability to flexibly chunk prefill requests, resulting in two critical limitations: (1) **Head-of-line blocking.** To process lengthy requests efficiently, systems must adopt large chunk sizes that monopolize compute resources, potentially delaying latency-sensitive requests and causing SLO violations. (2) **Limited Chunk-size Execution.** These systems fail to dynamically consolidate requests into larger chunks within iterations. This leads to small, inefficient chunks under unpredictable arrivals, causing substantial queuing delays for subsequent requests. Fig. 1 illustrates an example of these limitations. In (a), although Req 2 arrives later and has a stricter SLO, the system fails to preempt the current iteration of Req 1, resulting in Req 2 missing its deadline. In (b), despite Req 1’s small chunk size, the system fails to dynamically batch Req 2 once Req 1’s iteration has started. This inefficiency leads to GPU underutilization, ultimately extending Req 2’s queuing delay and causing it to miss the SLO target.

S1: Layer-level chunked prefill. For efficient prefill execution with multi-SLO compliance, we introduce the *layer-level chunked prefill* that includes two key operations: (1) preempting ongoing prefill computations at layer boundaries to prioritize critical requests; and (2) dynamically merging

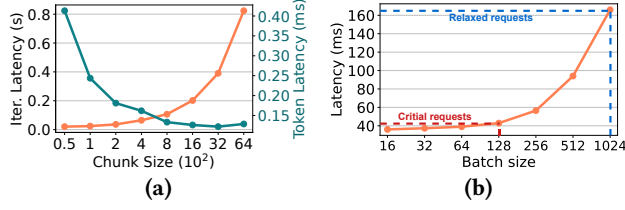


Figure 2. Serving performance when executing Qwen-14b model on an A100 GPU. (a) Latency of each iteration and average token processing latency under varying prefill chunk sizes. (b) Per-iteration latency under varying decode batch sizes. The sequence length per request is configured to 128.

requests into larger chunks within iterations to enhance resource utilization. This fine-grained mechanism allows prefill instances to maintain responsiveness to urgent incoming requests while simultaneously boosting throughput via larger chunks. As demonstrated in Fig. 1, in (a), with layer-level chunked prefill, the system can checkpoint Req 1's intermediate state after completing the current layer and promptly prioritize Req 2 when it detects that Req 1's remaining iteration time risks causing Req 2 to miss its SLO target. This swift transition ensures Req 2 meets its latency target through timely execution. In (b), the system determines that Req 2 should be merged with Req 1 to avoid SLO violations by fully exploiting the GPU capacity. To achieve this, it first processes the initial three layers of Req 2, then merges the remaining computations of both Req 1 and Req 2. This ensures both requests meet their SLOs.

L2: Unified decode batching. In the decode phase, the available batch size varies significantly for requests with different TBT requirements. As shown in Fig. 2b, latency-critical requests require low TBT to ensure quick responsiveness, necessitating small batch sizes of fewer than 128. In contrast, relaxed requests can tolerate up to 7× more concurrent requests than latency-critical ones. However, the iteration-level scheduler serves all requests with a unified TBT target. To meet the strictest SLO, it configures decode instances to adopt small batch sizes, disregarding the heterogeneous TBT requirements of others. This severely constrains the execution of remaining relaxed requests, ultimately compromising the overall SLO attainment.

Fig. 3 illustrates an example of serving requests with three distinct SLO targets. Each rectangle represents the per-layer execution timeline of an individual request, assuming a four-layer model. For each instance, the duration of the requests with the strictest SLO to execute four layers corresponds to the execution timeline of one iteration. To guarantee compliance with the strictest SLO1 requirement, each instance is configured to process two concurrent requests. For the more relaxed SLO2 target, set at twice the SLO1, the permitted batch size increases to six. However, as illustrated in Fig. 3(a), to meet the SLO1 target, the unified decode batching only allows each instance to process just one SLO1 and one SLO2

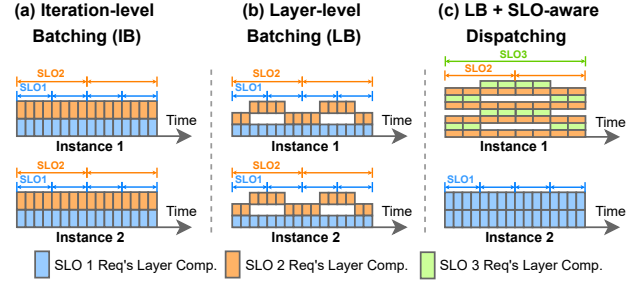


Figure 3. Comparison between different scheduling approaches in the decode phase.

request simultaneously. This bottleneck inevitably forces other SLO2 and SLO3 requests to miss their SLO targets.

S2: Layer-level decode batching. To achieve non-unified scheduling, we propose exploiting the tolerance of relaxed requests for higher latency to execute more requests concurrently. To realize this idea, we introduce *layer-level batching*, which batches requests at the granularity of individual layers. In each iteration, the instance scheduler determines the number of layers to execute for each request, while efficiently caching and restoring intermediate states for subsequent execution. This finer-grained control significantly boosts the available batch size of decode instances. As illustrated in Fig. 3(b), each SLO2 request is configured to process two layers per iteration. This scheduling decision allows the system to accommodate two additional SLO2 requests with SLO guarantees by interleaving the execution of SLO2 requests within iterations.

However, the underlying discrepancies in TBT requirements across requests still constrain the scalability of batch sizes. To address this issue, layer-level batching should be complemented with an SLO-aware dispatching strategy that prioritizes colocating requests with compatible SLOs. As illustrated in Fig. 3(c), assigning the stringent SLO1 requests to Instance 2 while batching the remaining requests in another one allows eight requests to execute simultaneously and meet their respective SLO targets. This underscores the need to jointly optimize request dispatching and layer-level batching to achieve system-wide efficiency.

4 System Overview

In this section, we detail the overall system architecture of Laser, as illustrated in Fig. 4. It mainly encompasses the following key components:

The *Global Controller* serves as the centralized orchestrator of Laser. To build latency models, it initiates an offline profiling process during system initialization to gather latency measurements from all serving instances, correlating performance with the number of input tokens and context length. At runtime, the *Global Controller* selects the prefill instance for new requests and employs a group-based algorithm to assign them to decode instances. These instance groups are

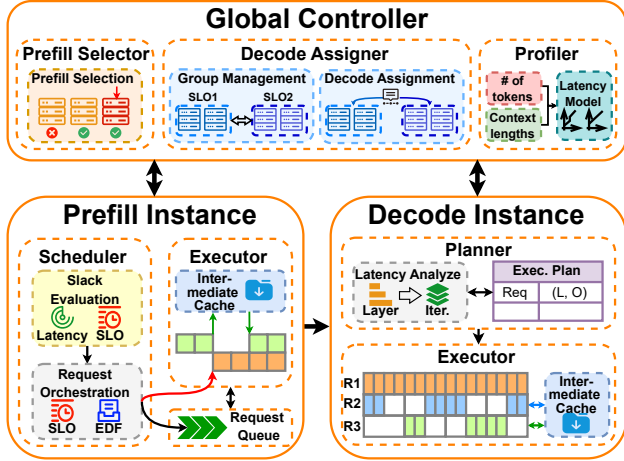


Figure 4. The architecture overview of Laser.

dynamically managed through a group management procedure, with each dedicated to accommodating requests with homogeneous TBT targets.

Each prefill instance executes the prefill phase of requests through the coordination of the *Scheduler* and the *Executor*. The *Scheduler* evaluates the latency slack of incoming requests, deciding whether to preempt the ongoing prefill requests to prioritize critical requests or increase chunk sizes. It also orchestrates queued requests based on the SLO targets and the earliest-deadline-first (EDF) principle. The *Executor* handles prefill computations and supports layer-level preemption and restoration. To ensure efficiency, states of preempted requests are checkpointed in an intermediate cache for seamless resumption.

Each decode instance executes the decode phase of requests, integrating the *Planner* and the *Executor*. The *Planner* dynamically analyzes per-iteration latency and constructs execution plans to guide decode execution by specifying two key parameters for each request: (1) the number of layers to execute (L) for each iteration, and (2) the scheduling offset (O) to balance workloads of different layers. The *Executor* then performs decoding computations accordingly, switching requests efficiently at layer boundaries. This is achieved through caching and restoration of request states via a dedicated intermediate cache.

5 Modular Latency Modeling

LLM architectures comprise two types of modules. Stateless modules, including the QKV projection, attention output projection, and feed-forward modules, process each input token independently without relying on previously generated tokens. In contrast, the self-attention module computes a weighted sum over all preceding key-value vectors in the context window, referred to as the stateful module. Laser separately models these modules to achieve precise latency estimation for each transformer layer.

For stateless modules, the latency remains stable within certain token ranges but spikes abruptly when the number of tokens exceeds specific thresholds due to the tile-quantization effect in GPU computation [6]. To capture this, Laser models the latency of stateless modules with a piecewise linear function:

$$\omega(n) = \begin{cases} a_0 \cdot n + b_0 & t \in [1, n_0) \\ \dots & \\ a_m \cdot n + b_m & t \in [n_{m-1}, n_m) \end{cases} \quad (1)$$

where n denotes the number of input tokens. Each linear segment i is determined by the slope a_i and intercept b_i , employing a fixed segment width of 32 tokens—a common divisor of standard GPU tile sizes. Additionally, the communication overhead from model parallelism techniques (e.g., tensor parallelism) increases linearly with the token count and is integrated into Equation (1) for precise modeling.

For the stateful self-attention module, latency is mainly determined by two factors: (1) **Total request context length**, which dominates computation and memory access overhead; and (2) **Token count**, which increases GPU thread block switching overhead during decode attention but improves parallelism during prefill attention. Our experiments show strong linear correlations between module latency and these parameters, with the absolute Pearson coefficients exceeding 0.78. Thus, the latency can be formulated as a linear function of the token count n and the total context length $\sum_{r=1}^n c_r$:

$$\tau(n, \sum_{r=1}^n c_r) = \alpha \cdot n + \beta \cdot \sum_{r=1}^n c_r + \gamma \quad (2)$$

Here, α , β , γ are related coefficients.

The aforementioned latency models jointly constitute the serving latency of a transformer layer, expressed as:

$$T(n, \sum_{r=1}^n c_r) = \omega(n) + \tau(n, \sum_{r=1}^n c_r) \quad (3)$$

To obtain the coefficients, Laser performs offline profiling by measuring module latencies across varying token counts and context lengths for each serving instance, and then fits these measurements to the corresponding latency models.

6 Intra-instance Request Scheduling

6.1 Layer-level Chunked Prefill

The *Scheduler* of each prefill instance performs layer-level chunked prefill by managing request chunking, preemption, and restoration at layer boundaries. Upon the arrival of a new request, the *Scheduler* computes the slack time as the gap between the TTFT target and the estimated prefill latency. If the remaining iteration time of the executing chunk exceeds the slack time, the *Scheduler* restores the the layer intermediate states of executing chunks and then: (1) forwards the new request to the same layer as the preempted chunk and merges them if doing so will not violate the new

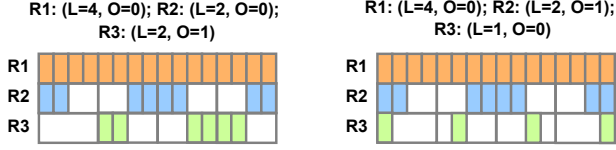


Figure 5. An example of request execution under different execution plans. The total number of model layers is 6.

request's SLO; or (2) directly prioritizes executing the new request if merging would risk an SLO violation. If preemption is not required, the *Scheduler* attempts to merge the new request into executing chunks when the request queue is empty, which indicates that no pending requests can be chunked with the executing one. Specifically, it computes the executing chunk's slack time, partitions the new request, and selects a chunk size that satisfies its SLO deadline. It then forwards the chunk to the layer of the executing one and merges them for subsequent prefill. If merging is infeasible, the new request is enqueued for later execution.

The *Scheduler* also strategically orchestrates the execution order of queued requests using the earliest-deadline-first (EDF) principle, minimizing SLO violations by prioritizing those with the most urgent deadlines. Specifically, it sorts queued requests by their arrival time plus their TTFT targets. After completing each prefill computation, the *Scheduler* dynamically chunks pending requests based on the tightest deadlines to maximize prefill throughput with SLO guarantees.

6.2 Layer-level Decode Batching

Within each decode instance, the *Planner* schedules decode requests at layer granularity to optimize decode throughput with SLO compliance. It constructs execution plans that determine two key parameters for each request: (1) L : the number of layers executed per iteration, and (2) O : the scheduling offset, which indicates the delay (in iterations) before the next execution. Figure 5 shows how L and O govern request execution across iterations under different plans.

6.2.1 Latency Analysis. The *Planner* implements a layer-wise analytical approach to evaluate per-iteration latency of a given execution plan. Specifically, for each request r , L_r and O_r denote its layer count and scheduling offset, respectively. Given the total number of layers N , the first execution of layer j for this request occurs at iteration $\lceil j/L_r \rceil + O_r$, and subsequent executions follow periodically every $\lceil N/L_r \rceil$ iterations. Thus, layer j is executed at iteration i only if $(i - \lceil j/L_r \rceil - O_r)$ is divisible by $\lceil N/L_r \rceil$. We denote this execution condition as a binary value $x(r, i, j)$, where $x(r, i, j) = 1$ indicates the request r would execute layer j at iteration i .

Given x , the *Planner* estimates the latency of layer j in iteration i as $T_d(\sum_{r \in R_d} x(r, i, j), \sum_{r \in R_d} x(r, i, j) \cdot c_r)$. Here, R_d is the set of requests assigned to decode instance d . The term $\sum_{r \in R_d} x(r, i, j)$ represents the batch size processed by

Algorithm 1: Execution Plan Construction

```

1 Function ExecPlan( $N, R, L, O, R^*$ ):
2   if  $R^* \neq \emptyset$  then
3      $L_r = N, O_r = 0$  for  $r \in R^*$ ;
4      $R = R \cup R^*$ ;
5    $Iter = LatencyEstimate(L, O)$ ;
6    $Tg = \min_{r \in R} SLO_r^{TBT}$ ;
7    $P = GroupRequests(R, L, O)$ ;
8   while True do
9     if  $Iter > Tg$  then
10       $r^{opt} = \arg \max_{L_r > \lceil \frac{N \cdot Tg}{SLO_r^{TBT}} \rceil} SLO_r^{TBT}$ ;
11       $L_{r^{opt}} = \lceil \frac{N \cdot Tg}{SLO_{r^{opt}}^{TBT}} \rceil$ ;
12       $O_{r^{opt}} = \arg \min_{O \in [0, M]} T(P[L_{r^{opt}}, O])$ ;
13       $Iter = LatencyEstimate(L, O)$ ;
14      if  $Iter < Tg$  then
15        break;
16    else
17       $r^{opt} = \arg \min_{L_r < N} SLO_r^{TBT}$ ;
18       $L_{r^{opt}} = N; O_{r^{opt}} = 0$ ;
19       $Iter = LatencyEstimate(L, O)$ ;
20      if  $Iter > Tg$  then
21        Restore the configuration of  $r^{opt}$ ;
22        break;
23  return  $L, O, R, Iter, Tg$ ;

```

layer j , while $\sum_{r \in R_d} x(r, i, j) \cdot c_r$ captures the total context length. The latency for the entire iteration i is obtained by aggregating the contributions of all N layers. As per-iteration latency varies significantly due to dynamic batch sizes and context lengths across layers, the *Planner* simulates several future iterations and takes the maximum predicted latency as the estimated per-iteration latency for the execution plan.

6.2.2 Execution Plan Construction. Building on the latency analysis, the *Planner* dynamically updates execution plans using real-time request metrics. This process poses two fundamental challenges. First, the intricate relationship between execution plans and per-iteration latency makes direct optimization intractable, while the vast configuration space, spanning layer counts and scheduling offsets for each request, creates a combinatorial explosion that rules out brute-force search. Second, growing request context lengths continually shift the optimal execution plan. However, reconstructing plans at each iteration introduces non-trivial scheduling overhead and induces latency fluctuations, increasing the risk of SLO violations.

To address these challenges, the *Planner* follows two key principles: (1) Maximizing the magnitude of changes on each

affected request to minimize the total number of updates and reduce disruptive reordering caused by scheduling offsets. (2) Triggering updates only during critical events, such as request arrivals or departures, or when per-iteration latency approaches the strictest request SLO. Guided by these principles, the *Planner* designs an efficient algorithm for plan construction, as outlined in Algorithm 1.

Initially, the *Planner* configures each newly arrived request $r \in R^*$ to execute all N layers every iteration without delay. It then estimates the current per-iteration latency and sets the TBT target Tg as the minimum requirement across requests. Requests are grouped based on their execution configurations. For example, $P[\frac{N}{2}, 1]$ denotes the group of requests that execute half of the layers per iteration with one-iteration delay.

Following this, the *Planner* iteratively adjusts execution plans to maintain per-iteration latency near Tg . If the estimated latency exceeds the target, the *Planner* selects the request r^{opt} with the most relaxed SLO and reduces the layer count to the minimum required for meeting r^{opt} 's SLO. This greedy adjustment maximizes the change magnitude and approaches optimality, as the largest throughput gains come from fully utilizing the least constrained resources (relaxed SLOs) first. The updated scheduling offset is set to balance the latency of request groups with the same layer count (line 12). Here, $T(P[\cdot])$ denotes the per-layer latency of request groups $P[\cdot]$. This process continues until the per-iteration latency falls within the target. Conversely, if the estimated latency is below the strictest SLO, the *Planner* prioritizes the request with the most stringent requirements, attempting to execute all layers per iteration. If this adjustment risks an SLO violation, the *Planner* reverts the configuration and terminates the update. The final updated plan and associated statistics are then output to direct the subsequent execution of requests.

7 Inter-instance Request Dispatching

The *Global Controller* dynamically routes new requests to phase-specific instances at runtime, applying tailored dispatching policies for prefill and decode phases to meet their respective colocation requirements.

7.1 Prefill Instance Selection

For prefill instances, colocating requests with diverse SLOs allows the system to exploit the latency slack of relaxed requests, thereby improving overall SLO compliance. To achieve this, the *Global Controller* continuously monitors the scheduling decisions and request statistics of each instance. Using this information, the *Global Controller* identifies which instances can admit the new request without violating TTFT SLOs under EDF scheduling and layer-level chunked prefill. Among the feasible instances, it selects the one with the largest latency slack, defined as the minimum gap between each request's estimated prefill time and its SLO deadline.

Algorithm 2: Group-based Decode Assignment

```

1 Function LatencyIncrement( $r^*, d$ ):
2   // Execute in decode instance  $d$ 
3   Instance  $d$  attains current  $N, R, L, O$ ;
4    $Iter = LatencyEstimate(L, O)$ ;
5    $L^*, O^*, R^*, Iter^*, Tg^* = ExecPlan(N, R, L, O, r^*)$ ;
6   if  $Iter^* > Tg^*$  or Memory Shortage then
7     return  $\infty$ ;
8   else
9     return  $\sum_{r \in R^*} Iter^* \cdot \lceil \frac{N}{L_r} \rceil - \sum_{r \in R} Iter \cdot \lceil \frac{N}{L_r} \rceil$ ;
10 Function DecodeAssignment( $G, r^*$ ):
11   for  $d \in \text{all decode instances}$  do
12      $I_d = LatencyIncrement(r^*, d)$ ;
13   Sort  $G$  based on  $|SLO_{g \in G} - SLO_{r^*}^{TBT}|$ ;
14   for  $g \in G$  do
15      $I^{opt}, d^{opt} = \min_{d \in g} I_d, \arg \min_{d \in g} I_d$ ;
16     if  $I^{opt} \neq \infty$  then
17       return  $d^{opt}$ ;
18   Assign  $r^*$  to the least-loaded instance;
```

If no instance can safely admit the request, the controller chooses the instance with the fewest prefill tokens and executes the request on a best-effort basis during idle periods.

7.2 Decode Request Assignment

In contrast, decode instances benefit from grouping requests with similar TBT SLOs, which reduces TBT discrepancies and improves batching efficiency. However, dispatching decode requests poses two major challenges. First, colocating similar-SLO requests causes severe resource imbalance across instances due to skewed application popularity and varying request lengths, while unpredictable arrivals further complicate the trade-off between utilization and batching efficiency. Second, assigning a new request requires evaluating its performance impact across all decode instances. Yet, with layer-level execution, complicated per-layer batch tracking and analysis make centralized performance evaluation in the *Global Controller* computationally prohibitive.

To address these challenges, the *Global Scheduler* employs two key designs: (1) **Instance group management**: It maintains dedicated instance groups for SLO-homogeneous requests, prioritizing intra-group assignments while permitting cross-group dispatching for load balancing. (2) **Decentralized performance evaluation**: New requests are pre-allocated to decode instances, enabling each instance to assess the impact of assignments locally. This decentralized approach minimizes central scheduling overhead.

7.2.1 Group-based assignment. Building on these designs, the *Global Scheduler* employs a group-based assignment algorithm, as shown in Algorithm 2. Given the instance groups G and an incoming request r^* , the *Global Scheduler* evaluates the impact of assigning r^* to each decode instance. Specifically, each instance first estimates its per-iteration latency before accepting the new request. It then applies the *ExecPlan* algorithm described in Algorithm 1 to generate an updated plan that incorporates r^* and recomputes the latency. If admitting r^* would cause SLO violations or memory exhaustion, the instance rejects it by returning ∞ ; otherwise, it returns the increment in aggregated TBT of all requests after accommodating r^* . With the increment statistics, the *Global Scheduler* sorts instance groups by the absolute difference between their designated SLO and r^* 's target. It then iterates through instance groups to select the instance that minimizes the TBT increment with SLO compliance. If no suitable instance is found, the request is assigned to the least-loaded instance for best-effort execution.

7.2.2 Instance Group Management. The *Global Scheduler* dynamically reassigns instances to different TBT groups to adapt to workload distributions. It derives the number of required instances for each group by dividing the arrival rate of requests by the given target by the per-instance throughput at the maximum SLO-compliant batch size. Instances are then allocated proportionally to the demand of each group. Notably, since instance groups are virtual constructs that affect only request assignment, the *Global Scheduler* can flexibly resize them in response to workload changes.

7.3 Implementing Laser at larger scales

Scaling Laser to large clusters faces two main challenges. First, although Laser employs decentralized performance evaluation across instances for efficient scheduling, scaling the number of serving instances inevitably increases scheduling overhead. This includes network overhead for coordinating statistics between the controller and instances, as well as computational overhead for request dispatching. These overheads can be alleviated by replicating the controller, with each replica independently managing a subset of instances. Second, resource exhaustion or sudden failures in serving instances can introduce stragglers, which degrade overall system efficiency. To address this, Laser sets timeout thresholds in the controller when monitoring prefill instance statistics and preallocating requests to decode instances, enabling the timely detection of potential failures in serving instances.

8 Implementation

Laser is an efficient LLM serving system designed to support multi-SLO workloads, incorporating vLLM [23] and Ray [5] as the default inference and communication backend.

Table 1. Average request lengths and SLO targets.

Dataset	Input	Output	TTFT	TBT
ShareGPT (SG)	350.9	187.9	0.25 s	100 ms
HumanEval (HE)	201.5	105.2	0.125 s	50 ms
LongBench (LB)	2084.5	95.7	2 s	200 ms

Intermediate state management. To enable efficient request switching, each instance maintains an intermediate cache in GPU memory (analogous to the KV cache) and a state manager for indexing active requests. The intermediate cache is allocated 16,384 tokens for prefill and 2,048 tokens for decode instances, consuming less than 256 MB of GPU memory for the Llama-70B model. Such a small cache size is sufficient for Laser because intermediate states are maintained at layer granularity, incurring minimal overheads. When the cache reaches capacity, Laser evicts the request with the least stringent SLO and asynchronously restores it once there is enough available cache space. Furthermore, Laser enhances the efficiency of switching intermediate states with a fused CUDA kernel that combines state caching and retrieval into a single operation.

Cache migration. Laser asynchronously migrates the KV cache between prefill and decode instances at layer granularity, overlapping this transfer with prefill computation. Since Laser operates without prior knowledge of requests, it handles memory exhaustion or SLO violations caused by dynamic context lengths by reassigning decode requests with live cache migration [34], ensuring continuous token generation during transfer.

9 Evaluation

9.1 Experiment Setup

Cluster Setup. We deploy Laser on a local cluster consisting of four physical hosts, each equipped with four NVIDIA A100 80GB GPUs. The hosts are interconnected via a 100 Gbps LAN, while GPUs within each host are linked via NVLink.

Models. We evaluate Laser with Qwen2.5-14B, Qwen2.5-32B [22], and LLaMA-3-70B [15] models. Serving instances are configured with one-, two-, and four-way tensor parallelism, respectively, depending on the model size.

Metrics. Following prior work [43], we focus on two key metrics: SLO attainment and goodput, which is defined as the throughput achieved while maintaining 90% SLO attainment. A decode request violates its SLO if its P99 TBT exceeds the SLO target [6].

Workloads. We choose three different LLM applications for evaluation and configure the SLO target according to prior works [6, 26, 43]. Specifically, we use the ShareGPT dataset [1] for the chatbot application [38], the HumanEval dataset [10] for the code completion task [11], and the LongBench dataset [8] for the document summarization [2]. Table 1 summarizes the workload characteristics, including average

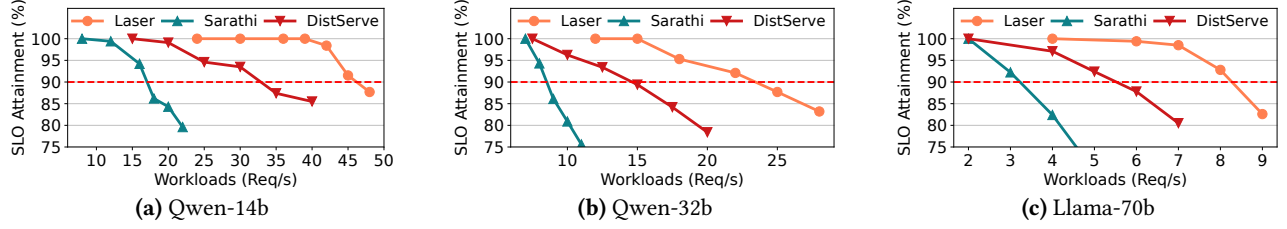


Figure 6. SLO attainment under varying workloads.

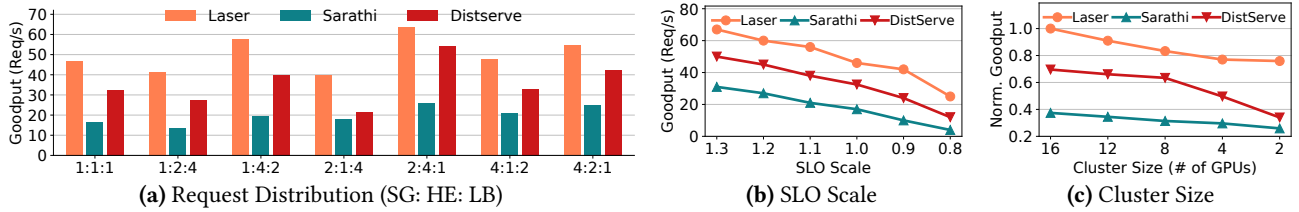


Figure 7. Serving goodput under varied experimental configurations.

request lengths and baseline SLO targets. To emulate real-world user behavior, we generate request arrivals using a Poisson distribution with varying rates.

Baselines. To demonstrate the effectiveness of Laser’s design, we compare it against state-of-the-art LLM serving systems, Sarathi-serve [6] and Distserve [43]. Sarathi-serve adopts a prefill-decode aggregation architecture and incorporates the chunked prefill mechanism to prevent head-of-line blocking caused by long prefill computations. In contrast, Distserve disaggregates the prefill and decode phases and applies iteration-level scheduling in each serving instance. Both baseline systems adopt the earliest-deadline-first (EDF) scheduling policy instead of the first-come-first-served (FCFS) policy for fair comparison.

9.2 End-to-end Evaluation

We first evaluate the end-to-end performance of Laser under different workload intensities. As shown in Fig. 6, compared to Sarathi-Serve and DistServe, Laser significantly reduces the SLO violation rate and improves goodput by up to 68.9% and 1.67 $\times$, respectively. When the attainment target of goodput rises to 99%, throughput improvement further increases to 1.85 $\times$ over all baselines. Sarathi-serve applies decode-oriented scheduling that prioritizes the execution of decode requests. To meet the strict 50 ms TBT target, it enforces a limited prefill chunk size for each instance, leading to significant prefill SLO violations. Distserve addresses this limitation by disaggregating the prefill and decode phases onto dedicated instances. This allows each phase to meet its specific requirements, enhancing the goodput by 80.5%. However, its reliance on iteration-level scheduling struggles to accommodate the heterogeneous demands of multi-SLO workloads, resulting in performance degradation. Laser employs layer-level scheduling for each phase, allowing latency-critical requests to execute promptly while batching more relaxed

ones for higher efficiency. Furthermore, it incorporates inter-instance request dispatching with phase-tailored policies for system-wide optimization. Consequently, Laser boosts goodput by 43.4% on Qwen-14B, 68.9% on Qwen-32B, and 56.6% on Llama-70B, compared to all baseline schemes.

To further assess the robustness of Laser, we evaluate it with the Qwen-14b model under varied configurations:

Request distribution. To understand the impact of heterogeneous requests, we assess the goodput under different request distributions. As illustrated in Fig. 7a, the improvement of Laser grows as the proportion of relaxed requests increases. Specifically, when the distribution shifts from 2:4:1 to 2:1:4, where relaxed requests dominate the distribution, Laser’s improvement grows from 19.4% to over 86%. This significant gain is attributable to two advantages: First, Laser effectively exploits the latency slack of relaxed requests to optimize both prefill chunking and decode batching, leading to higher utilization of GPU capabilities. Second, it can preempt lengthy prefill computations (e.g., from LongBench) to prioritize the latency-critical requests (e.g., from HumanEval), thereby drastically reducing SLO violations.

SLO scale. To assess Laser’s sensitivity to SLO targets, we conduct experiments by varying the SLO scale of requests relative to the baseline target depicted in Table 1. As shown in Fig. 7b, Laser’s advantage becomes more pronounced as the SLO tightens, thanks to its ability to efficiently execute critical requests without compromising others. For a tight 0.8 $\times$ SLO, Laser achieves goodput gains of up to 1.08 $\times$ and 6.25 $\times$ against Distserve and Sarathi-serve, respectively. Moreover, even under a relaxed 1.3 $\times$ SLO, Laser maintains a goodput improvement of over 34%, demonstrating its robustness across a broad spectrum of SLO targets.

Cluster size. The available GPU resources in a cluster determine the total number of instances, which in turn influences request dispatching decisions. To investigate this impact, we deploy instances with varying GPU allocations and report

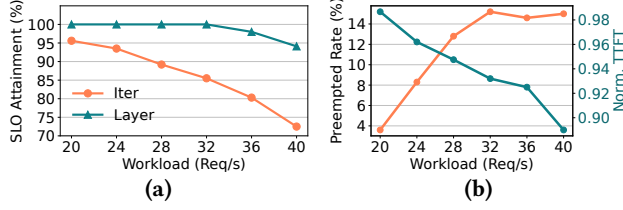


Figure 8. (a) Comparison between layer-level chunked prefill (Layer) and iteration-level (Iter) chunked prefill. (b) Preemption rate and TTFT (normalized to the iteration-level) of layer-level chunked prefill.

the goodput, which is normalized by the result of Laser with all 16 GPUs. As depicted in Fig. 7c, the normalized goodput of all schemes declines with smaller cluster sizes, as the reduced scale limits the performance gains achievable through optimized request dispatching. Laser’s advantage over the strongest baseline, Distserve, exhibits a two-stage trend. The improvement first declines from 43.4% to 31.3% as smaller clusters constrain the group-based assignment of Laser to mitigate heterogeneous TBT demands. However, once the cluster size drops below 8 GPUs, Distserve fails to mitigate head-of-line blocking from lengthy requests, whereas Laser prioritizes critical requests at layer boundaries, boosting SLO attainment and achieving a $1.23\times$ goodput gain. On the other hand, Laser maintains a consistent improvement over Sarathi-serve, proving its robustness across all cluster sizes.

9.3 Ablation Study

In this section, we provide an in-depth analysis of the individual contributions of Laser’s components using the Qwen-14b model. Each phase is configured with 8 instances.

Prefill Phase. We quantify the performance gain of layer-level chunked prefill by comparing it against an iteration-level baseline. As depicted in Fig. 8a, the implementation of layer-level chunked prefill significantly reduces SLO violations for prefill requests, with the benefit amplifying under higher loads—the violation rate drops from 4.4% at 25 Req/s to 21.6% at 50 Req/s. The underlying reason can be explained in two aspects. First, enabling request switching at layer boundaries allows the system to prioritize and execute latency-critical requests in time. As Fig. 8b demonstrates, the preemption rate in layer-level chunked prefill increases with workload intensity, enabling more critical requests to meet their deadlines. Second, this approach supports the flexible consolidation of new requests into larger chunks per iteration, reducing the average TTFT by over 10%. Together, these advances lead to substantial improvements in SLO attainment.

Decode Phase. We evaluate the improvement of Laser in the decode phase by decomposing its innovations: layer-level decode batching and group-based decode assignment. We compare it against two baselines: one using iteration-level batching (Iter) and another using only layer-level batching

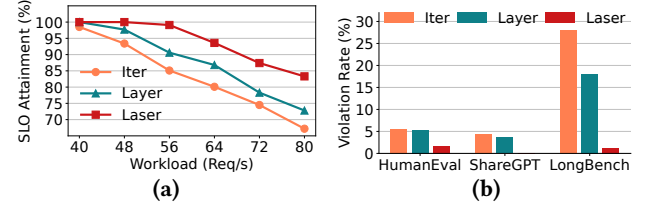


Figure 9. (a) Comparison between layer-level batching (Layer), iteration-level (Iter) batching, and Laser. (b) The violation rate of each request type at 56 req/s.

(Layer). As illustrated in Fig. 9a, layer-level batching effectively exploits the SLO slack of relaxed requests to increase the capacity of decode instances in serving them. Thus, it lowers the TBT SLO violation rate by more than 6.7% compared to the iteration-level baseline. Laser builds on this foundation and integrates a group-based decode assignment policy that prioritizes collocating requests with similar SLO targets to mitigate the discrepancies. This synergistic approach further reduces the SLO violation rate by an additional 10.5%. To better understand these benefits, we analyze the SLO violation rate for each request type under a workload of 56 req/s. As shown in Fig. 9b, the violation rate of relaxed requests (from ShareGPT and LongBench) consistently decreases with the integration of layer-level batching and group-based assignment, demonstrating their effectiveness in handling heterogeneous SLO requirements. We also evaluate the benefits of Laser using another key decode metric, Time-Per-Output-Token (TPOT), which measures the average time required to generate each output token. The SLO target for TPOT is set to the same value as TBT, as defined in Table 1. Compared to iteration-level batching, Laser reduces the TPOT SLO violation rate by up to 11.8%, demonstrating its superior performance across both decode metrics. This improvement is driven by Laser’s ability to concurrently execute more decode requests, which minimizes unnecessary request queuing delays.

9.4 Prediction Accuracy

We evaluate the prediction accuracy of Laser’s modeling methodology, encompassing individual modules and the aggregated latency of each layer and iteration under layer-level scheduling. As shown in Fig. 10a, Laser delivers high prediction accuracy ranging from 94.8% to 98.6% for both stateful and stateless modules across various models. This high module-level accuracy directly enables precise prediction of aggregated per-layer and per-iteration latency. Specifically, as depicted in Fig. 10b, Laser maintains accuracy of over 94.6% for these aggregated predictions across all batch sizes, demonstrating robust performance under complex layer-level scheduling. Notably, the entire profiling process for constructing latency models completes in under two seconds, adding negligible overhead to instance initialization.

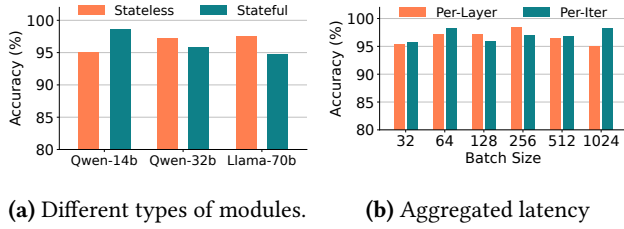


Figure 10. Prediction accuracy of Laser.

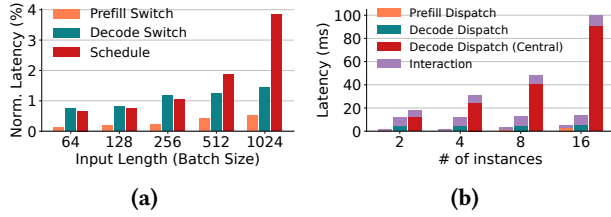


Figure 11. (a) Normalized latency of request switching and intra-instance scheduling under varying input lengths (batch sizes) in the prefill (decode) phase. (b) Latency of inter-instance request dispatching.

9.5 Overhead Analysis

Layer-level scheduling. Implementing the layer-level scheduling requires fine-grained intra-instance scheduling and switching the intermediate states of requests at layer boundaries. To measure the overhead, we profile the corresponding latency under varying input lengths in the prefill phase and batch sizes in the decode phase. Fig. 11a depicts the latency of request switching and scheduling, normalized by the request execution time under corresponding settings. Request switching introduces negligible overhead ($<1.5\%$) for both prefill and decoding, even under large batch sizes and long sequences. Scheduling exhibits a slightly higher overhead of up to 3.8% due to the complex latency estimation and fine-grained execution plan updates required to meet SLO guarantees. However, this cost is mitigated in two ways: first, plan construction is triggered only by critical events, allowing its cost to be amortized over tens of iterations. Second, and more importantly, Laser completely masks this overhead by overlapping plan construction with model execution, thereby isolating the decode computation from any performance impact.

Inter-instance dispatching. To demonstrate the scalability of the *Global Controller*, we evaluate the request dispatching overhead under different numbers of serving instances, with the batch size of each instance fixed to 512. As illustrated in 11b, the *Global Controller* efficiently selects prefill instances within 2 ms upon request arrival. For decode requests, the dispatching overhead is approximately 10 ms, primarily originating from the inherent controller-instance interaction (Interaction). Notably, Laser’s dispatching overhead remains nearly constant, regardless of the number of

instances. This is achieved by delegating performance evaluation to the instances themselves rather than the *Global Controller*. To validate this design choice, we also measure the overhead when the *Global Controller* centrally evaluates decode execution performance during request dispatching (Central). In this case, the overhead is substantially higher and grows with the number of instances, underscoring the importance of parallelizing performance evaluation.

10 Related Work

Prefill-Decode Disaggregation. The prefill-decode disaggregated architecture has been widely explored for serving LLMs [13, 16, 20, 25, 28, 30, 32, 43]. Splitwise [28] leverages heterogeneous devices to meet the differing resource demands of each phase and dynamically switches instance roles to adapt to workload fluctuations. Distserve [43] independently optimizes parallelism strategies and resource allocation for each instance type based on workload distribution and network bandwidth. Mooncake [30] employs a centralized scheduler to manage KV caches across instances, reusing them to reduce redundant prefill computations. Despite the effectiveness in meeting individual SLO targets for each phase, existing disaggregated systems are unaware of the challenges introduced by multi-SLO workloads, leading to significant degradation in serving goodput and frequent SLO violations.

LLM Serving with SLO guarantees. To satisfy the stringent SLO requirements of LLM serving, recent works have proposed various strategies to improve SLO attainment and system throughput [6, 14, 19, 21, 24, 27, 29, 33]. Sarathi-Serve [6] introduces chunked prefill, which partitions prefill computation across multiple iterations to mitigate head-of-line blocking from long requests, thereby preserving the SLO attainment of decode requests. SOLA [19] adopts a state-aware scheduling strategy to dynamically switch the scheduling preference between prefill-prioritized and decode-prioritized at the iteration level. DynamoLLM [33] tunes the GPU frequency, resource allocation, and model parallelism of each application to enhance energy efficiency with SLO compliance. Nevertheless, existing approaches fail to provide fine-grained control over per-iteration computation with layer-level scheduling to mitigate requirement discrepancies between requests.

11 Conclusion

We propose layer-level scheduling, a novel mechanism that schedules requests at the granularity of individual layers to handle heterogeneous SLO requirements. To implement this mechanism, we introduce Laser, a serving system that efficiently coordinates intra-instance scheduling with inter-instance dispatching to optimize serving efficiency while ensuring multi-SLO guarantees. Comprehensive evaluations demonstrate that Laser improves both SLO attainment and

serving goodput over state-of-the-art systems, highlighting that effective request scheduling should extend beyond iteration-level granularity to handle complex workloads.

Acknowledgements

We sincerely thank the anonymous reviewers of PPoPP'26 and our shepherd, Myeongjae Jeon, for their insightful suggestions. This work was supported in part by the Guangdong Key Area R&D Program under Grant No. 2025B0101080001, and the National Natural Science Foundation of China under Grants No. 62432004 and 62172454.

References

- [1] 2023. ShareGPT. <https://sharegpt.com/>.
- [2] 2025. LangChain Summarize Text. <https://python.langchain.com/docs/tutorials/summarization/>.
- [3] 2025. NVIDIA Dynamo Platform. <https://www.nvidia.com/en-us/ai/dynamo/>.
- [4] 2025. Qwen FinAgent. <https://www.tongyi.com/discover?type=FindAgent>.
- [5] 2025. Ray. <https://docs.ray.io/en/latest/index.html>.
- [6] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in llm inference with sarathi-serve. *arXiv preprint arXiv:2403.02310* (2024).
- [7] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Merouane Debbah, Etienne Goffinet, Daniel Heslow, Julien Launay, Quentin Malartic, Badreddine Noune, Baptiste Pannier, and Guilhermo Penedo. 2023. Falcon-40B: an open large language model with state-of-the-art performance. (2023).
- [8] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2023. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508* (2023).
- [9] Marc Brysbaert. 2019. How many words do we read per minute? A review and meta-analysis of reading rate. *Journal of memory and language* 109 (2019), 104047.
- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [12] Siyuan Chen, Zhipeng Jia, Samira Khan, Arvind Krishnamurthy, and Phillip B Gibbons. 2025. SLOs-Serve: Optimized Serving of Multi-SLO LLMs. *arXiv preprint arXiv:2504.08784* (2025).
- [13] Shiyang Chen, Rain Jiang, Dezhi Yu, Jinlai Xu, Mengyuan Chao, Fanlong Meng, Chenyu Jiang, Wei Xu, and Hang Liu. 2024. KVDirect: Distributed Disaggregated LLM Inference. *arXiv preprint arXiv:2501.14743* (2024).
- [14] Ke Cheng, Zhi Wang, Wen Hu, Tiannuo Yang, Jianguo Li, and Sheng Zhang. 2025. SCOOT: SLO-Oriented Performance Tuning for LLM Inference Engines. In *Proceedings of the ACM on Web Conference 2025*. 829–839.
- [15] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.
- [16] Jingqi Feng, Yukai Huang, Rui Zhang, Sicheng Liang, Ming Yan, and Jie Wu. 2025. WindServe: Efficient Phase-Disaggregated LLM Serving with Stream-based Dynamic Scheduling. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1283–1295.
- [17] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [19] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. [n. d.]. SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling. In *Eighth Conference on Machine Learning and Systems*.
- [20] Cunhen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. 2024. Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads. *arXiv preprint arXiv:2401.11181* (2024).
- [21] Jinqi Huang, Yi Xiong, Xuebing Yu, Wenjie Huang, Entong Li, Li Zeng, and Xin Chen. 2025. SLO-Aware Scheduling for Large Language Model Inferences. *arXiv preprint arXiv:2504.14966* (2025).
- [22] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- [23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [24] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, et al. 2025. AdaServe: SLO-Customized LLM Serving with Fine-Grained Speculative Decoding. *arXiv preprint arXiv:2501.12162* (2025).
- [25] Yunkai Liang, Zhangyu Chen, Pengfei Zuo, Zhi Zhou, Xu Chen, and Zhou Yu. 2025. Injecting Adrenaline into LLM Serving: Boosting Resource Utilization and Throughput via Attention Disaggregation. *arXiv preprint arXiv:2503.20552* (2025).
- [26] Zizhao Mo, Jianxiong Liao, Huanle Xu, Zhi Zhou, and Chengzhong Xu. 2025. Hetis: Serving LLMs in Heterogeneous GPU Clusters with Fine-grained and Dynamic Parallelism. *arXiv preprint arXiv:2509.08309* (2025).
- [27] Bowen Pang, Kai Li, and Feifan Wang. 2025. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching. *arXiv preprint arXiv:2503.05248* (2025).
- [28] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [29] Archit Patke, Dharmath Reddy, Saurabh Jha, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2025. Hierarchical Autoscaling for Large Language Model Serving with Chiron. *arXiv preprint arXiv:2501.08090* (2025).
- [30] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A kv-cache-centric disaggregated architecture for llm serving. *arXiv preprint arXiv:2407.00079* (2024).
- [31] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.
- [32] Gursimran Singh, Xinglu Wang, Ivan Hu, Timothy Yu, Linzi Xing, Wei Jiang, Zhefeng Wang, Xiaolong Bai, Yi Li, Ying Xiong, et al. 2024.

- Efficiently serving large multimedia models using EPD Disaggregation. *arXiv preprint arXiv:2501.05460* (2024).
- [33] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1348–1362.
 - [34] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 173–191.
 - [35] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. 2016. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2818–2826.
 - [36] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
 - [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
 - [38] Guan Wang, Sijie Cheng, Xianyu Zhan, Xiangang Li, Sen Song, and Yang Liu. 2023. Openchat: Advancing open-source language models with mixed-quality data. *arXiv preprint arXiv:2309.11235* (2023).
 - [39] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
 - [40] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
 - [41] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
 - [42] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).
 - [43] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. *arXiv preprint arXiv:2401.09670* (2024).

Received 2025-08-31; accepted 2025-11-10